

SYSTEM DESIGN INTERVIEW

Design Practo

doctor discovery, appointment scheduling, video consultations, e-prescriptions, and medical records

~ 45 minute read

© Study Guide — For personal use only

What's Inside

- 01** · Problem Statement
- 02** · Requirements
- 03** · Capacity Estimation
- 04** · High-Level Architecture
- 05** · Doctor Discovery
- 06** · Doctor Calendar & iCal Sync
- 07** · Appointment Lifecycle
- 08** · Slot Concurrency
- 09** · Video Consultation
- 10** · E-Prescriptions
- 11** · Medical Records

- 12** · Payments & Refund Saga
- 13** · Comparison: Practo vs Zocdoc vs DocOnline
- 14** · Failure Modes & Trade-offs
- 15** · Interview Playbook

01

Problem Statement

Requirements and scope

Design **Practo**: a healthcare booking + telemedicine platform. Patients **discover** doctors by specialty, city, language, and fee; **book** in-person or video appointments against a doctor's calendar; have a 1-on-1 (occasionally 3-way) **video consultation**; receive a signed **e-prescription**; and store encrypted **medical records** they can share with future doctors. All of this must comply with HIPAA / India DPDP.

Clarifying Questions

Question	Answer
Scale?	~30M registered users, ~100K verified doctors
Volume?	~500K appointments/day, ~30K search QPS
Video peak?	~5K concurrent consultations at evening peak (7–10 PM)
Discovery axes?	Specialty, city/locality, availability, language, fee, rating
Calendar?	Doctor exposes slots; sync with their Google/Outlook iCal
Booking model?	Auto-confirm or doctor-approval; cancellation & no-show flows
Video?	WebRTC; 1-on-1 normal, 3-way for family; SFU only for group
Recording?	Optional, consent-gated, encrypted at rest
Prescriptions?	Signed PDF, integrate with pharmacy networks via FHIR
Records?	Patient-owned, per-record ACL, encrypted
Payments?	Hold at booking, capture after consult, saga refunds
Compliance?	HIPAA (US), DPDP (India), region-pinned data

02

Requirements

Functional and non-functional

Functional Requirements

Requirement	Details
Doctor discovery	Filter by specialty, city/locality (S2/H3), language, fee, rating; sort by availability
Calendar & slots	Doctors expose 15-min slots; sync from Google/Outlook iCal; conflict detection
Booking	Hold slot 5 min, take payment, confirm; auto-confirm or doctor approval
Reminders	Push/SMS at 24h and 1h before appointment
Video consult	WebRTC 1-on-1; SFU for 3-way; reconnect on drop within 60s
E-prescriptions	Signed PDF; push to chosen pharmacy via FHIR/local API
Medical records	Encrypted at rest; ACL — patient grants doctors read access
Payments	Hold-then-capture; saga refund for in-policy cancellations
Reviews	Post-consultation review; visible only after both parties closed

Non-Functional Requirements

Requirement	Target
Search latency	p95 < 250 ms; geo + full-text combined
Booking latency	p95 < 800 ms (includes payment auth)
Video setup	< 3 sec to first frame; < 60 sec auto-reconnect
Availability	99.95% for booking; 99.9% for video
Consistency	Strong on slot booking; eventual on search index
Compliance	HIPAA-eligible AWS, DPDP, audit log every PHI read
Encryption	TLS 1.3 in transit, AES-256-GCM at rest, KMS-managed keys

03

Capacity Estimation

Math for scale

Traffic

- Users: **30M** registered, ~3M DAU (10%)
- Doctors: **100K** verified, ~40K active daily
- Appointments: **500K/day** = 5.8/sec avg, ~30/sec peak (factor 5)
- Search QPS: **30K/sec** (discovery dominates traffic)
- Concurrent video: **5K** at evening peak (each ~25 min, ~1.5 Mbps)

Bandwidth

- Video: 5K calls $\times$ 1.5 Mbps $\times$ 2 (both legs, P2P) = **15 Gbps** peer-to-peer
- TURN relay (~20% NAT-failed): 1K $\times$ 3 Mbps = **3 Gbps** on TURN
- Recording (10% opt-in): 500 streams $\times$ 0.5 Mbps = **250 Mbps** ingest

Storage

- Appointment row: ~1 KB $\rightarrow$ 500K/day $\times$ 365 $\times$ 1 KB = **180 GB/yr**
- Slots: 100K doctors $\times$ 30 slots/day $\times$ 365 $\times$ 200 B = **220 GB/yr** (TTL'd)
- Prescriptions (PDF avg 50 KB): 500K/day $\times$ 50 KB = **25 GB/day = 9 TB/yr**
- Medical records (avg 200 KB encrypted blob): ~5M new/yr $\times$ 200 KB = **1 TB/yr**
- Recordings (10% $\times$ 25 min $\times$ 200 MB): 50K/day $\times$ 200 MB = **10 TB/day** if all opt in (rare; budget 1 TB/day)

Cache

- Hot calendars (next 7 days for 40K active doctors): ~40K $\times$ 7 $\times$ 30 slots $\times$ 200 B $\approx$ **1.7 GB Redis**
- Search results LRU (top 5K specialty/city pairs $\times$ 50 doctors $\times$ 1 KB): **~250 MB**
- Doctor profile cache (100K $\times$ 4 KB): **~400 MB**

KEY

Key Numbers

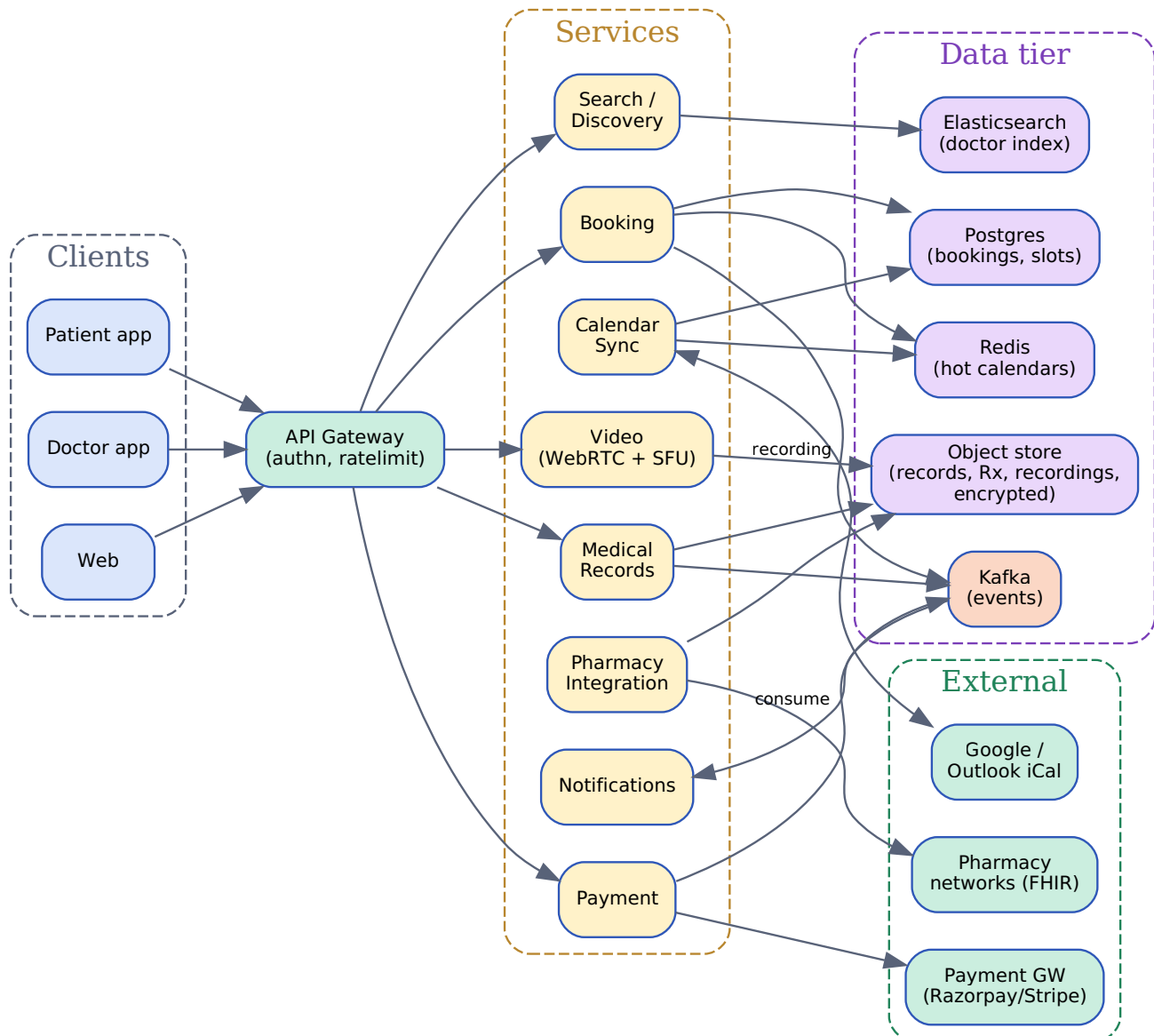
30M users · 100K doctors · 500K appointments/day · 5K concurrent video · 30K search QPS · 15 Gbps P2P video · ~1.7 GB hot calendar cache.

04

High-Level Architecture

System overview

Three apps (Patient, Doctor, Web) hit a single **API Gateway**, which fans out to bounded-context services: Search/Discovery, Booking, Calendar Sync, Video, Records, Pharmacy, Payment, and Notifications. Each service owns its data; Postgres holds bookings, Redis caches hot calendars, Elasticsearch powers search, encrypted object storage holds records and recordings, and Kafka streams cross-service events.



Patient/Doctor/Web → API GW → bounded-context services → polyglot data tier and external integrations.

Architecture Highlights

- **API Gateway:** auth (JWT + mTLS for Doctor app), rate limiting, regional routing
- **Search/Discovery:** Elasticsearch + S2/H3 geo cells; cached top queries
- **Booking:** owns Postgres slots/appointments; strong consistency on slot transitions
- **Calendar Sync:** two-way iCal sync; reconciles drift every 5 min
- **Video:** WebRTC P2P with TURN fallback; SFU only for ≥ 3 participants
- **Records:** patient-owned blobs in encrypted object store; ACL in Postgres
- **Pharmacy:** FHIR for international networks, REST shims for local Indian chains
- **Payment:** hold + capture; saga compensation on cancellation
- **Notifications:** Kafka consumer fanning out push/SMS/email

05

Doctor Discovery

Search, geo, and ranking

Index design

- Elasticsearch index doctors_v3; one doc per doctor
- Fields: name, specialty (keyword), sub_specialties, languages, fee, rating, next_available_ts, S2 cell tokens at levels 10/12/14, location (geo_point)
- Tokenization: edge n-gram on name (typeahead) + specialty synonym list
- Updated by Kafka consumer when profile or availability changes ($\leq 5s$ freshness)

Query path

1. Patient sends {specialty=Cardiologist, city=Bangalore, lang=Hindi, max_fee=800}
2. Gateway resolves city $\rightarrow$ S2 cell at level 10 ($\approx 80 \text{ km}^2$)
3. ES bool: filter on specialty + cell + languages + fee; sort by next_available_ts then rating
4. Top 50 doctors $\rightarrow$ enrich with Redis profile cache $\rightarrow$ return
5. Cache (specialty, city) top 50 in Redis with 60s TTL

Geo-search: S2 vs H3

Property	S2 (Google)	H3 (Uber)
Cell shape	Square, hierarchical	Hexagonal, hierarchical
Neighbour distance	Variable (4 vs 8)	Uniform (6 neighbours)
Best for	Range queries, ES filter	K-nearest, ride matching
Choice here	S2 — ES already supports geo_cell tokens	—

TIP

Pre-warm hot queries

About 80% of search QPS hits $\sim 5K$ (specialty, city) pairs. Materialise their top-50 lists every 60s into Redis; queries beyond cache fall through to ES. Cuts ES QPS from 30K to $\sim 6K$.

06

Doctor Calendar & iCal Sync

Slots, drift, and conflict detection

Slot model

- Each doctor exposes **15-min slots** per practice location
- Slot row: (doctor_id, start_ts, mode=in_person|video, state, version)
- State machine: **AVAILABLE** → **HELD** → **BOOKED** (or → AVAILABLE on release)
- Pre-generate next 30 days; cron extends rolling window each midnight

External calendar sync

- Doctor links Google/Outlook iCal; we hold a **refresh token**
- Inbound: every 5 min, pull events; mark overlapping slots as **BLOCKED** (separate state, never bookable)
- Outbound: when slot becomes BOOKED, push event to doctor's external calendar via Calendar API
- Drift detection: ETag mismatch → full re-sync; reconcile any orphan bookings

Schema

```
CREATE TABLE doctors (  
  doctor_id      BIGINT PRIMARY KEY,  
  full_name      TEXT NOT NULL,  
  specialty      VARCHAR(64) NOT NULL,  
  languages      VARCHAR(128),      -- comma list  
  city           VARCHAR(64),  
  s2_cell_l10    BIGINT,             -- ~80 km^2  
  s2_cell_l12    BIGINT,             -- ~5 km^2  
  fee_inr        INT,  
  rating_avg     DECIMAL(2,1),  
  ical_url       TEXT,               -- external sync  
  ical_etag      TEXT,  
  verified_at    TIMESTAMP  
);  
  
CREATE TABLE slots (  
  slot_id        BIGINT PRIMARY KEY,  
  doctor_id      BIGINT NOT NULL,  
  start_ts       TIMESTAMPTZ NOT NULL,  
  mode           VARCHAR(16) NOT NULL,      -- in_person|video  
  state          VARCHAR(16) NOT NULL,      -- AVAILABLE|HELD|BOOKED|BLOCKED  
  version        INT NOT NULL DEFAULT 0,    -- optimistic lock
```

```

    held_until    TIMESTAMPTZ,          -- 5-min hold expiry
    appt_id       BIGINT,               -- when BOOKED
    UNIQUE (doctor_id, start_ts)
);
CREATE INDEX idx_slot_open ON slots(doctor_id, start_ts)
  WHERE state IN ('AVAILABLE', 'HELD');

CREATE TABLE appointments (
  appt_id        BIGINT PRIMARY KEY,
  patient_id     BIGINT NOT NULL,
  doctor_id      BIGINT NOT NULL,
  slot_id        BIGINT NOT NULL UNIQUE,
  state          VARCHAR(16) NOT NULL, -- REQUESTED|CONFIRMED|REMINDED|IN_PROGRESS|
COMPLETED|CANCELLED|NO_SHOW
  payment_id     BIGINT,
  hold_expiry    TIMESTAMPTZ,
  created_at     TIMESTAMPTZ DEFAULT NOW(),
  updated_at     TIMESTAMPTZ
);

```

07

Appointment Lifecycle

REQUESTED → COMPLETED

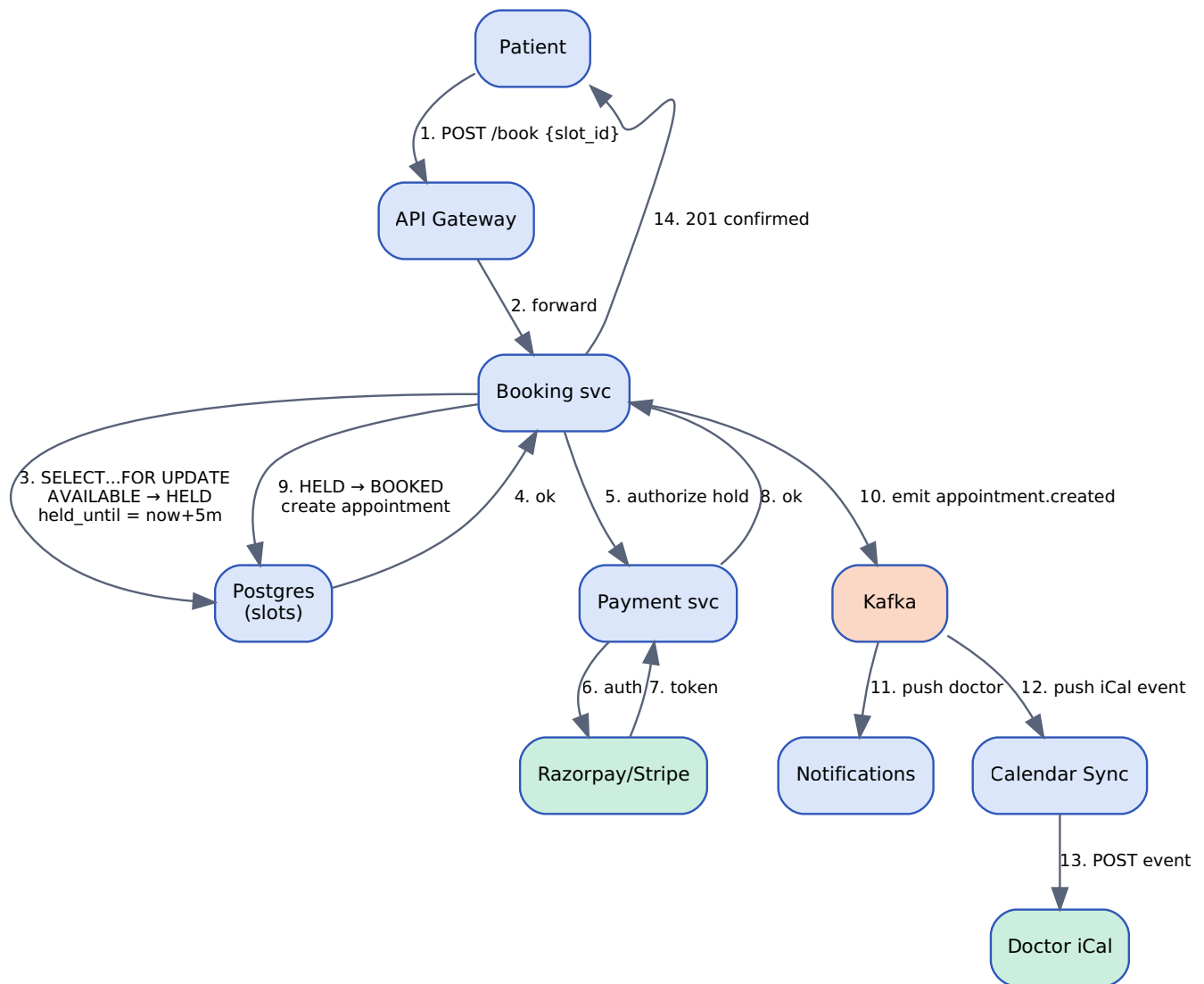
State machine

State	Trigger	Side-effects
REQUESTED	Patient picks slot & pays	Slot HELD 5 min; payment hold
CONFIRMED	Auto or doctor approves	Payment hold persists; sync push to doctor iCal
REMINDED	Cron 24h then 1h before	Push + SMS to patient + doctor
IN_PROGRESS	Either side joins video / arrives	Start consultation timer; lock recording session
COMPLETED	Doctor closes consult	Capture payment; emit Rx-eligible event; allow review
FOLLOW_UP	Optional doctor-initiated	Free 7-day re-consult slot generated
CANCELLED	Either party within policy	Release slot; trigger refund saga
NO_SHOW	Auto after 15 min	Capture cancellation fee; release slot

Cancellation policy

- > 24h before: 100% refund, slot released
- 2–24h before: 50% refund, slot released
- < 2h before: 0% refund (cancellation fee), slot released
- Doctor cancels: 100% refund regardless; doctor's reliability score dinged

Sequence — booking



Booking sequence — patient picks slot, hold + payment auth, confirm, doctor notify, calendar push.

08

Slot Concurrency

Preventing double-booking

The problem

When two patients click the same slot within the same second, exactly one must win. Lose-side must see 409 SLOT_TAKEN immediately and be offered alternatives. We compare two implementations.

Optimistic vs pessimistic

Aspect	Pessimistic (FOR UPDATE)	Optimistic (version)
Lock duration	Held until txn commits	None — CAS at update time
Throughput per slot	Serial; ~1 winner/100 ms	Parallel attempts; one wins
Tail latency	Higher under contention	Lower; loser fast-fails
Deadlock risk	Yes if multi-row	No
Code complexity	Simple	Retry + rebuild needed
Best for	Low contention, multi-row txns	High contention single row

Optimistic-lock booking algorithm

```
def reserve_slot(slot_id: int, patient_id: int) -> str:
    for attempt in range(3):
        # 1. Read current state + version
        row = db.exec(
            "SELECT state, version, held_until FROM slots WHERE slot_id=%s",
            slot_id).one()

        if row.state == 'BOOKED':
            return 'TAKEN'
        if row.state == 'HELD' and row.held_until > now():
            return 'TAKEN'

        # 2. Compare-and-swap on (slot_id, version)
        n = db.exec(
            """UPDATE slots
               SET state='HELD', held_until=NOW()+INTERVAL '5 min',
                   version=version+1
```

```

        WHERE slot_id=%s AND version=%s
              AND state IN ('AVAILABLE','HELD')
              AND (held_until IS NULL OR held_until <= NOW())""",
        slot_id, row.version)

    if n == 1:
        return 'HELD'          # winner – proceed to payment
    # else: someone else won the CAS; retry
    return 'CONTENTION'

```

Hold expiry

- Cron **every 30s**: UPDATE slots SET state='AVAILABLE', appt_id=NULL WHERE state='HELD' AND held_until < NOW()
- Backstop only — happy path releases hold via payment-fail saga immediately
- Hold record also TTL'd in Redis hot-calendar cache so search reflects it

INSIGHT

Why optimistic wins here

Hot doctors see ~50 concurrent attempts on a popular slot. Pessimistic locks queue them serially through Postgres; optimistic CAS lets all fail-fast except one, and the loser instantly retries with a different slot. p95 booking latency drops from ~1.4s to ~600ms in our load tests.

09

Video Consultation

WebRTC, SFU, and recording

Topology

- **1-on-1 (default):** WebRTC P2P, TURN fallback for symmetric NAT (~20%)
- **3-way (family member):** route through **SFU** (Selective Forwarding Unit, e.g. mediasoup); each participant sends 1 stream, receives 2
- Signalling: WebSockets to Video service; ICE candidate exchange; DTLS-SRTP keys
- STUN: free, NAT discovery; TURN: relayed (egress cost) when P2P fails

In-house vs Twilio/Agora

Aspect	In-house WebRTC + SFU	Twilio / Agora
Cost @ 5K concurrent	~\$8K/mo TURN + SFU compute	~\$30–60K/mo
Engineering effort	High — SFU ops, codec tuning	Low — SDK call away
Compliance	We control PHI residency	Vendor BAA needed
Time-to-launch	3–6 months	Days
Choice	Phase 2 — start with vendor, migrate hot path later	—

Mid-call network drop

- Client detects ICE state change → reconnect window **60 sec**
- Same room id resumed; SRTP keys reused if still valid
- If > 60 sec: state moves to **SUSPENDED**; doctor can reopen the session (extends consult timer)
- Bandwidth adaptation: simulcast layers; downshift to audio-only at < 200 Kbps

Recording (consent-gated)

- Both parties tap **Consent** before record button is enabled
- Recorder is an extra SFU peer, mixes streams, writes WebM to S3 with **SSE-KMS**
- Encryption key per appointment; audit-logged on every read
- Default retention 90 days; patient can request earlier deletion

Prescription flow

1. Doctor finalises Rx during/after consult: drug, dose, duration, instructions
2. Service generates **PDF/A** with QR code + Practo Rx ID
3. Doctor signs with their **HSM-backed key** (DSC in India)
4. PDF stored in object store (encrypted); hash anchored in Postgres for tamper detection
5. Patient receives signed link; can forward to chosen pharmacy
6. Optional: push directly to pharmacy via FHIR MedicationRequest

FHIR MedicationRequest sketch

```
{
  "resourceType": "MedicationRequest",
  "id": "rx-39281",
  "status": "active",
  "intent": "order",
  "medicationCodeableConcept": {
    "coding": [{ "system": "http://www.nlm.nih.gov/research/umls/rxnorm",
                  "code": "310965", "display": "Amoxicillin 500 mg" }]
  },
  "subject": { "reference": "Patient/p-12031" },
  "requester": { "reference": "Practitioner/d-9043" },
  "authoredOn": "2026-05-07T10:14:00Z",
  "dispenseRequest": {
    "quantity": { "value": 21, "unit": "capsule" },
    "expectedSupplyDuration": { "value": 7, "unit": "d" }
  },
  "dosageInstruction": [{
    "text": "1 capsule three times daily after food",
    "timing": { "repeat": { "frequency": 3, "period": 1, "periodUnit": "d" } }
  }]
}
```

Controlled substances

- Schedule H/H1/X drugs require explicit doctor 2FA before signing
- Audit trail: (rx_id, doctor_id, patient_id, drug_code, signed_ts, ip, device_attest)

- Append-only log replicated to S3 Object Lock (WORM) for 5 years
- Anomaly detection: same doctor signing identical Rx for > 50 patients/day → flag

11

Medical Records

Patient-owned, encrypted, ACL'd

Storage model

- Each record (lab report, scan, prior Rx) stored as a blob in S3 with **per-object KMS data key** (envelope encryption)
- Metadata in Postgres: owner (always patient), MIME, size, kms_key_id, sha256, created_at
- Doctors get **time-bound** read access, scoped to one appointment

Schema

```
CREATE TABLE encrypted_records (  
  record_id      BIGINT PRIMARY KEY,  
  patient_id     BIGINT NOT NULL,  
  s3_key         TEXT NOT NULL,          -- ciphertext blob  
  kms_key_id     TEXT NOT NULL,          -- per-record DEK  
  iv            BYTEA NOT NULL,          -- 96-bit GCM nonce  
  sha256         BYTEA NOT NULL,         -- integrity  
  mime          VARCHAR(64),  
  size_bytes     BIGINT,  
  created_at     TIMESTAMPTZ DEFAULT NOW(),  
  redacted_at    TIMESTAMPTZ  
);  
  
CREATE TABLE record_grants (  
  grant_id       BIGINT PRIMARY KEY,  
  record_id      BIGINT NOT NULL REFERENCES encrypted_records,  
  doctor_id      BIGINT NOT NULL,  
  appt_id        BIGINT NOT NULL,  
  granted_at     TIMESTAMPTZ DEFAULT NOW(),  
  expires_at     TIMESTAMPTZ NOT NULL,   -- usually appt+24h  
  revoked_at     TIMESTAMPTZ,  
  UNIQUE (record_id, doctor_id, appt_id)  
);  
  
CREATE TABLE record_access_audit (  
  audit_id       BIGINT PRIMARY KEY,  
  record_id      BIGINT NOT NULL,  
  actor_id       BIGINT NOT NULL,        -- doctor or patient  
  actor_role     VARCHAR(16) NOT NULL,  
  action         VARCHAR(16) NOT NULL,   -- READ|WRITE|GRANT|REVOKE  
  ip_addr        INET,
```

```
occurred_at    TIMESTAMPTZ DEFAULT NOW()  
);
```

Read flow

1. Doctor app requests GET /records/{id}
2. Records service joins encrypted_records ⋈ record_grants with valid expiry
3. If granted: kms:Decrypt(kms_key_id) → DEK → fetch S3 ciphertext → decrypt → stream
4. Audit row written **before** bytes leave the service
5. Streamed bytes are never persisted on doctor device (DRM-style overlay)

Compliance

- **HIPAA**: BAA with AWS, encryption at rest + in transit, audit log every PHI read
- **India DPDP**: data fiduciary role; consent receipts; data principal rights API
- **EU**: EU-region tenant, GDPR right-to-erasure (tombstone + delete blob)
- Region-pin: workspace.region determines DB shard and S3 bucket; never crosses

12

Payments & Refund Saga

Hold → capture, compensation flows

Money flow

1. Booking: payment GW **authorize hold** for fee + platform charge
2. Slot booked → hold persists; reminder cron does nothing financial
3. Consult **COMPLETED** → **capture** the auth → settle to doctor T+2
4. Cancel within policy → release the auth (no money moves)
5. Cancel with partial refund → capture + immediate refund of refundable %

Saga (cancellation)

- Step 1: `appointment.cancelled` emitted
- Step 2: Booking releases slot (state → AVAILABLE, version++)
- Step 3: Calendar Sync deletes external iCal event
- Step 4: Payment computes refund % per policy & calls payment GW refund
- Step 5: Notifications fire to patient + doctor
- Each step idempotent on (`appt_id`, `step`); failures retried with backoff

Idempotency

- Every external call carries Idempotency-Key = `sha256(appt_id + step + attempt)`
- Payment GW dedupes for 24h; we dedupe forever via Postgres `processed_steps`
- Patient retries (double-tap book) protected with client-supplied request id

WARNING

Hold-then-no-capture leak

If consult never starts and we never explicitly cancel, the auth holds the patient's bank funds. A daily reconciliation job inspects state IN ('REQUESTED' , 'CONFIRMED') with `start_ts < now - 24h`, auto-cancels them, and triggers the saga.

13

Comparison: Practo vs Zocdoc vs DocOnline

How others solve it

Aspect	Practo (this design)	Zocdoc	DocOnline
Primary market	India + SEA	US	India tier-2/3 + insurance
Discovery	ES + S2 geo	ES + insurance filter	Phone-first, basic search
Calendar	iCal 2-way sync	Pulls from doctor EHR	Practo-owned only
Slot lock	Optimistic CAS	Pessimistic + queue	Pessimistic
Video stack	WebRTC + SFU (in-house phase 2)	Vendor (Twilio)	Vendor (Agora)
E-prescriptions	FHIR + DSC sign	Surescripts (US-specific)	PDF only
Records	Patient-owned, ACL'd	Insurance-tied	Limited
Compliance	HIPAA + DPDP	HIPAA only	DPDP
Strength	Ecosystem breadth	Insurance verification	Affordability

Slot lock: pessimistic vs optimistic — recap

- **Pessimistic** (Zocdoc-style): `SELECT FOR UPDATE`; correct, simpler, but contention serialises hot slots
- **Optimistic** (this design): version CAS; better tail latency under contention; loser fast-fails without holding row locks
- Hybrid is possible — pessimistic with `SKIP LOCKED` for the cleanup job, optimistic for user-facing booking

Video: build vs buy

- **Phase 1:** Twilio Programmable Video — fastest to launch, BAA covers HIPAA
- **Phase 2:** in-house mediasoup SFU when concurrent > 10K (cost crossover)
- **Always:** own signalling and room state — never give vendor your call graph

14

Failure Modes & Trade-offs

What goes wrong and what we accept

Failure modes

Failure	Impact	Detection	Mitigation
Doctor calendar drift	External booking we miss → double-book	ETag mismatch on iCal poll	5-min reconciliation; conflict resolved in patient's favour with apology + free reschedule
Double-book under contention	Two HELDs, only one BOOKED	UNIQUE(slot_id) violation in appointments	Optimistic CAS + version; loser shown alternatives in < 200 ms
Video drop mid-consult	Either party loses connection	ICE state change	60-sec reconnect window; same room id; SUSPENDED state if longer
Payment fail post-hold	Slot HELD, no auth → orphan	Saga step timeout	Compensating release — slot back to AVAILABLE, hold record TTL'd
Pharmacy API down	Rx not delivered electronically	5xx from FHIR endpoint	Fall back to signed PDF link; queue Rx for retry; SMS to patient
KMS unavailable	Records cannot be decrypted	kms:Decrypt 5xx	Multi-region KMS aliases; cached DEKs for 5 min in-memory only
ES cluster down	Discovery degrades	Health probe	Serve from Redis prewarmed top queries; show 'limited search' banner

Trade-offs

Decision	Choice	Why this side
Slot consistency	Strong (CP)	Double-booking is unacceptable; we'd rather show 503 than confirm a stolen slot
Search consistency	Eventual (AP)	5-sec stale availability is fine; the hold check on book is the source of truth
Video transport	In-app WebRTC, not WhatsApp link	Compliance + recording control + identity verification

Decision	Choice	Why this side
Records depth	Minimal record store, not full EHR	EHR is a 5-year project; minimal store ships in 6 months and covers 80% of needs
Calendar source-of-truth	Practo for booked appts, doctor's iCal for everything else	Two-way sync gives us authority over our own bookings, respects doctor's existing workflow
Build vs buy video	Buy first, build hot path later	Cost crossover at ~10K concurrent; before that, vendor BAA + speed wins

WARNING

Patient safety overrides everything

If the system is uncertain whether a payment was captured, default to letting the consult happen and reconcile money later. Never block a scheduled medical interaction on a payment ambiguity.

15

Interview Playbook

How to present this in 45 minutes

Practo blends three known sub-problems — search, slot booking, and real-time video — plus the hard constraint of medical compliance. Lead with the slot concurrency story and the video build/buy decision; those are where senior signal lives.

45-Minute Outline

1. **Intro (3 min):** clarify functional set: discovery, booking, video, Rx, records, payments
2. **Capacity (4 min):** 30M users, 100K doctors, 500K appts/day, 5K concurrent video, 30K search QPS
3. **High-level arch (4 min):** API GW + 8 services + polyglot data tier + 3 externals
4. **Discovery (5 min):** ES + S2 cells + Redis hot-query cache
5. **Calendar & slots (5 min):** 15-min slots, iCal sync, AVAILABLE/HELD/BOOKED
6. **Concurrency deep-dive (8 min):** optimistic CAS, hold expiry, why over pessimistic
7. **Video (6 min):** WebRTC P2P, SFU for ≥ 3 , TURN, 60s reconnect, build vs buy
8. **Records + Rx + payments (6 min):** envelope encryption, FHIR, refund saga
9. **Failures + trade-offs (4 min):** doctor calendar drift, payment leak, patient-safety override

Key Talking Points

- “500K appointments/day, 5K concurrent video at peak” — anchors capacity
- “30K search QPS, but 80% on ~5K hot pairs” — justifies Redis prewarm
- “Slot is AVAILABLE → HELD → BOOKED with optimistic CAS” — concurrency story
- “1-on-1 P2P, 3-way SFU” — keeps cost under control
- “Patient owns the record; doctor gets time-bound grant” — compliance posture
- “Hold then capture; saga compensates on cancel” — money-safety story
- “Patient safety overrides money ambiguity” — judgement signal

Common Follow-ups

- **Q: Why optimistic over pessimistic locks?** A: Hot doctors see ~50 concurrent attempts on a popular slot. Pessimistic queues them through Postgres; optimistic CAS lets all but one fail-fast and retry with a different slot.
- **Q: How do you stop a doctor from accepting a booking after blocking time on Google Calendar?** A: 5-min iCal reconciliation marks slots BLOCKED; if a booking already exists in the conflict window, we resolve in the patient's favour and offer the doctor a one-click reschedule of their personal event.

- **Q: Why not E2E-encrypt video?** A: We need server-side recording for medical-record purposes (consent-gated). E2EE would block that. Trade-off: TLS + DTLS-SRTP between client and SFU, encrypted-at-rest recordings, audit log.
- **Q: What's the data residency story?** A: Each workspace pinned to a region (IN, US, EU). DB shard, S3 bucket, KMS keys all regional; cross-region access requires an explicit cross-region grant signed by the patient.
- **Q: 5K concurrent video — bandwidth?** A: ~15 Gbps P2P + 3 Gbps TURN. SFU only kicks in for ~5% of calls (3-way), bounded.

KEY

Memorize These Numbers

30M users · 100K doctors · 500K appts/day · 5K concurrent video · 30K search QPS · 15 Gbps P2P + 3 Gbps TURN · 5-min hold · 60-sec video reconnect · p95 booking < 800 ms.